Static Content Audit Field Guide

Unofficial technical reference inspired by Canadian public-safety report style.

Prepared	local workstation triage and defensive file review.
Classification	Personal technical working copy

Important notice: This is not an official RCMP document and is not affiliated with any police service or government agency. It is a defensive technical guide using a sober field-report style.

Purpose

This guide explains how to turn `static-content-audit.sh` reports into useful evidence leads using short, pasteable shell pipelines.

The goal is not to prove that a file is safe. The goal is to separate a large noisy file tree into a smaller review set: scripts, event handlers, executable bits, external references, embedded data, suspicious strings, and package ownership.

Primary tool:

```
/home/u/arbitrarily_useful/static-content-audit.sh
```

Typical report location:

```
/home/u/static-content-audits/
```

Operating Model

Treat the output like a triage notebook:

- Counts tell you where to look.
- Category blocks show sample paths.
- Line context shows why a file matched.
- Package ownership helps separate system defaults from local oddities.
- Manual review confirms whether a match matters.

The audit script is read-only except:

```
static-content-audit.sh exec-bits --fix
```

That mode only removes executable bits from matched files.

First Run

Run a root SVG audit and capture the report path:

```
r=$(/home/u/arbitrarily_useful/static-content-audit.sh svg /)
printf '%s\n' "$r"
```

Open the report:

```
less "$r"
```

Print only nonzero category counts:

```
awk -F= '/^[a-zA-Z0-9_]+=/ && $2+0 > 0 {print}' "$r"
```

Extract every path mentioned in the report:

```
rg -o '/[^:[:space:]]+' "$r" | sort -u
```

Avoid the Multiline Regex Trap

Do not split quoted regex across lines in zsh. This breaks:

```
rg '^\[(script_tag|event_handler_attr|
  foreign_object)\]$' report.txt
```

Use a pattern file instead:

```
printf '%s\n' \
'^\[script_tag\]$' \
'^\[event_handler_attr\]$' \
'^\[javascript_or_vbscript_uri\]$' \
'^\[external_href_http_file_ftp_script\]$' \
'^\[foreign_object\]$' \
'^\[entity_decl\]$' \
'^\[external_dtd\]$' \
'^/' > /tmp/svg-interest.patterns

rg -f /tmp/svg-interest.patterns "$r"
```

Fast rough version:

```
rg 'script_tag|event_handler|javascript_or_vbscript|external_href|foreign_object|entity_decl|external_dtd|^/' "$r"
```

Pull One Category Block

Extract paths under `[script_tag]`:

```
awk '/^\[script_tag\]/{f=1; next} /^\[/{f=0} f && /^\// {print}' "$r"
```

Extract paths under `[foreign_object]`:

```
awk '/^\[foreign_object\]/{f=1; next} /^\[/{f=0} f && /^\// {print}' "$r"
```

Make it reusable:

```
cat > /tmp/svg-block <<'EOF'
#!/usr/bin/env bash
cat="$1"
report="$2"
awk -v c="[$cat]" '$0 == c {f=1; next} /^\[/ {f=0} f && /^\// {print}' "$report"
EOF
chmod +x /tmp/svg-block

/tmp/svg-block script_tag "$r"
/tmp/svg-block foreign_object "$r"
```

Attach Package Ownership

For Arch or CachyOS packages:

```
awk '/^\[script_tag\]/{f=1; next} /^\[/{f=0} f && /^\// {print}' "$r" |
  xargs -r pacman -Qo
```

For `foreignObject` files:

```
awk '/^\[foreign_object\]/{f=1; next} /^\[/{f=0} f && /^\// {print}' "$r" |
  xargs -r pacman -Qo
```

For unowned paths:

```
rg -o '/[^:[:space:]]+' "$r" | sort -u |
while IFS= read -r p; do
  pacman -Qo -- "$p" >/dev/null 2>&1 || printf 'unowned: %s\n' "$p"
done
```

High-Interest SVG Categories

Start with these:

- `script_tag`
- `event_handler_attr`
- `javascript_or_vbscript_uri`
- `external_href_http_file_ftp_script`
- `foreign_object`
- `entity_decl`
- `external_dtd`
- `image_or_feimage_external_or_data`
- `cdata`
- `long_base64_like_blob`

Count them:

```
awk -F= '
/^(script_tag|event_handler_attr|javascript_or_vbscript_uri|external_href_http_file_ftp_script|foreign_object|entity_decl|ex
ternal_dtd|image_or_feimage_external_or_data|cdata|long_base64_like_blob)=/ {
  if ($2+0 > 0) print
}' "$r"
```

Decode Embedded Data

Find embedded `data:` references:

```
awk '/^\[data_href\]/{f=1; next} /^\[/{f=0} f && /^\// {print}' "$r"
```

Extract a base64 payload from one SVG:

```
svg=/path/to/file.svg
rg -o 'base64,[A-Za-z0-9+/=]+' "$svg" |
  sed 's/^base64,//' |
  base64 -d > /tmp/svg-embedded.bin

file /tmp/svg-embedded.bin
strings -a /tmp/svg-embedded.bin | head -80
```

Decode a CDATA-looking base64 blob:

```
svg=/path/to/file.svg
sed -n 's/.*<!\[CDATA\[//; s/\]\]>.*//p' "$svg" |
  base64 -d 2>/dev/null > /tmp/svg-cdata.bin

file /tmp/svg-cdata.bin
strings -a /tmp/svg-cdata.bin | head -80
```

Search decoded content for command or network markers:

```
strings -a /tmp/svg-cdata.bin |
  rg -i 'https?://|file://|javascript:|/bin/|bash|sh -c|curl|wget|python|perl|exec|eval'
```

Review Script-Bearing SVGs

Show exact line context:

```
rg -n -i '<script|</script>|onload=|javascript:' /path/to/file.svg
```

View a small safe window around a match:

```
sed -n '320,350p' /path/to/file.svg
```

Common benign cases:

- Documentation flamegraphs.
- Inkscape examples.
- Empty placeholder script tags.
- Local editor polyfills.

Higher-interest case:

- A script-bearing file inside an active icon theme or desktop theme.

Executable Bits

Audit executable SVGs:

```
/home/u/arbitrarily_useful/static-content-audit.sh exec-bits --ext svg /
```

Remove executable bits from SVGs in a specific tree:

```
/home/u/arbitrarily_useful/static-content-audit.sh exec-bits --ext svg --fix /home/u/.local/share/icons
```

Verify:

```
find / -xdev -type f -iname '*.svg' -perm /111 -print 2>/dev/null | wc -l
```

Strings Profiles

List available profiles:

```
/home/u/arbitrarily_useful/static-content-audit.sh list-patterns
```

Run a web-focused string scan:

```
s=$(/home/u/arbitrarily_useful/static-content-audit.sh strings --profile web /path/to/file-or-dir)
less "$s"
```

Run a broader suspicious scan:

```
s=$(/home/u/arbitrarily_useful/static-content-audit.sh strings --profile all --max-hits 50 /path/to/target)
```

Show hit counts:

```
awk '/^Hit counts by category/{f=1; next} /^$/{f=0} f' "$s"
```

Pretty-print TSV hits:

```
awk -F'\t' '/^[a-zA-Z0-9_]+\t/ {
  printf "%-24s line %-6s %s\n", $1, $3, $4
}' "$s"
```

Show only secret-like hits:

```
awk -F'\t' '$1 ~ /(secret|token|private|password|aws|github|bearer|jwt)/' "$s"
```

Combined Suspicious Sweep

For a folder or file:

```
d=$(/home/u/arbitrarily_useful/static-content-audit.sh suspicious /path/to/target)
printf '%s\n' "$d"
```

The folder contains:

```
00-run.log
01-inventory.txt
02-exec-bits-all.txt
03-svg.txt
04-refs.txt
05-strings-all.txt
```

Pull the important lines from all reports:

```
rg '^[a-zA-Z0-9_]+=|^Hit counts by category|script_tag|event_handler|javascript|private_key|token|foreign_object|external_dt
d' "$d"/*.txt
```

Quick Assessment Ladder

Low concern:

- Metadata-only matches.
- Documentation examples.
- Internal `xlink:href="#id"` references.
- Package-owned files with common upstream patterns.

Medium concern:

- `foreignObject` in active theme files.
- Big base64 blobs in UI assets.
- External DTD declarations.
- `data:` images where embedded content is not obvious.

High concern:

- `javascript:` or `vbscript:`.
- Event handlers in active UI assets.
- Script-bearing SVGs inside active icon or desktop themes.
- Unowned files in system-looking paths.
- Executable bits on data files.
- Decoded payloads containing shell commands, downloaders, or network callbacks.

Notes From Current Workstation SVG Audit

Observed root-filesystem SVG findings:

```
svg_files=137757
script_tag=6
event_handler_attr=1
javascript_or_vbscript_uri=0
external_href_http_file_ftp_script=6
data_href=175
foreign_object=16
doctype=1084
entity_decl=60
external_dtd=351
```

Observed `char-white` findings:

```
char_white_svg=9957
script_tag=0
event_handler_attr=0
javascript_or_vbscript_uri=0
external_href_http_file_ftp_script=0
data_href=0
foreign_object=3
doctype=0
entity_decl=0
external_dtd=0
```

Assessment: `char-white` showed poor package metadata and editor leftovers, but no active script indicators in the static checks. The highest-interest SVG from the earlier review was the BeautyLine `colab.svg` icon because it lives inside an icon theme and contains an embedded script.

Closing Standard

Do not call a file safe because it passed pattern scans. State the result precisely:

```
No script tags, event handlers, JavaScript URIs, external hrefs, or XML entity declarations were found by the static audit.
```

That is defensible. It is not the same as:

```
This file is safe.
```